DeepEcho: Classical Image Processing and Deep Learning for Echocardiographic Left-Ventricular Volume Estimation

Francesca Righetti[1,2], Riccardo Gibello[1,2], and Enrico Gianluca Caiani[1,2,3]

[1]DEIB, Politecnico di Milano, Milan, Italy
[2]D-Hygea Lab, Politecnico di Milano, Milan, Italy
[3]IRCCS Istituto Auxologico Italiano, San Luca Hospital, Milan, 20149, Italy

October 23, 2026

1 Context

Echocardiography is non-invasive, real-time, and widely available, but it is also the noisiest routine cardiac imaging modality. Factors like speckle, acoustic shadowing, and operator-dependent acquisition often make image quality the biggest limitation when extracting clinical measurements.

Your group will build a clinical predictor and investigate how image preprocessing and anatomical segmentation impact its downstream performance. The workflow consists of:

- **Mandatory Core:** Using classical image-processing methods to clean the images and reduce noise.
- **Optional Component:** Using deep learning methods to obtain anatomical segmentations that support the clinical predictor.

Data & Objectives

You will work with *still frames* captured at end-diastole (ED). Your objective is to implement a preprocessing pipeline that produces a clean, usable image and, where relevant, isolates the target anatomical structures. For example, calculating ventricular volume is a geometric task that becomes easier and more accurate once the heart cavity is clearly delineated.

2 Distributed Material

- **In-house Clinical Dataset.** ∼800 examinations, one ED frame per apical view (A2C, A4C). Distributed only in synthetically degraded form, using the same degradation procedure as the perturbed CAMUS images. Each frame is degraded independently, including the two views of the same patient, with a random combination of noise types at varying intensities. A patient contributes several frames: *grouping is by patient, never by frame.* Each frame is associated with an end-diastolic volume in mL, plus a binary low/high label obtained using sex-specific clinical cut-offs (166 mL for males, 124 mL for females) derived from the literature [1].

- **CAMUS Dataset [2]:** This dataset contains echocardiograms from 500 patients in 2-chamber (A2C) and 4-chamber (A4C) views, with 450 of them featuring public expert annotations at end-diastole (ED) and end-systole (ES). These annotations include pixel-level segmentations for the left ventricular (LV) cavity, myocardium, and left atrium, alongside ground-truth reference LV volumes for each patient. To make your pipeline more robust, you can use both the clean and synthetically perturbed versions (generated with the same procedure as the in-house frames) of the CAMUS images for data augmentation, and you can even generate additional perturbed images using the provided `noise_generation.ipynb` notebook.
- **Degradation generator** (`noise_generation.ipynb`). The notebook is provided to introduce controlled ultrasound-like perturbations into clean CAMUS images. During training, groups may combine clean and perturbed CAMUS examples as data augmentation, exposing the model to a broader range of image appearances. The aim is to improve robustness when the model is later applied to the in-house images, whose preprocessing may not completely remove acquisition artefacts or variability. **Its parameter ranges are illustrative and do not match those used to generate the distributed data.**

3 Objectives

Every group implements the same three parts:

(i) a **mandatory classical front-end**, justified on the basis of ultrasound physics;
(ii) a **fixed downstream predictor**, held constant across conditions;
(iii) a **controlled comparison** of preprocessing strategies under identical splits, including a no-processing baseline, a mandatory classical processing pipeline and, optionally, a deep learning segmentation/denoising model trained on CAMUS or an explainability pipeline.

	A — EDV volume (regression)	**B — EDV level (classification)**
Views	A4C + A2C	
Target	EDV, mL	EDV, low/high
Mandatory	Classic image preprocessing (denoising, contrast enhancement, etc.)	
Deep Learning (Optional)	LV cavity segmentation / Image denoising trained on CAMUS	
CAMUS used for	LV cavity contours; clean + perturbed images for augmentation / denoising	
Metrics	MAE, RMSE	ROC-AUC, PR-AUC, sensitivity/specificity

Project Steps

- **Classical Volume Estimation (Mandatory):**
  1. **Noise Identification & Image Restoration:** Every A2C and A4C frame has been processed with a random combination of noise types (possibly none) at varying intensities. Use the analytical methods covered in lectures (e.g., histogram analysis, clustering) to identify the specific noise present in each frame. Once identified, apply classical image processing techniques to clean the image and restore its original quality.
  2. **Embedding & Downstream Prediction:** Generate feature embeddings for your processed frames using a domain-pretrained backbone (e.g., EchoCLIP [14] or PanEcho [15]). Using these embeddings, train one of the following models:

  - A small classification head to categorize the End Diastolic Volume into "low" and "high" classes.
  - A small regression head to directly estimate the continuous End Diastolic Volume.

- **Optional: Deep Learning Segmentation/Denoising**
  Each group can add to their previously developed model one of the two following extensions as a preprocessing step:
  - **Deep Learning Segmentation.** Train a U-Net [3,4] on CAMUS end-diastolic (ED) LV cavity annotations and apply it to the clinical images after classical preprocessing stage; assess its impact on the downstream EDV task. To improve robustness to differences between CAMUS and the clinical images, compare (a) training on clean CAMUS only vs. (b) training using a combination of clean and synthetically perturbated CAMUS images.
  - **Deep Learning Denoising.** Using the provided clean/degraded frame pairs, train a compact denoiser (e.g., DnCNN [16] with DeepInverse [17]) to reverse it. Every frame of every CAMUS sequence is usable, not just the annotated ED frames. Substitute this denoiser for the classical filter and compare. Report both (i) PSNR/SSIM on held-out CAMUS and (ii) the effect on downstream classification accuracy.
- **Optional: Explainability.** Apply an attribution method (e.g. Grad-CAM [13]) to a few correctly and incorrectly predicted cases, and assess qualitatively whether the highlighted regions are anatomically plausible or driven by artifacts.

4 Protocol

- **Justify the operators.** The choice of classical filters must follow from the physics of ultrasound formation, not from trial and error.
- **Splits first.** Fix patient-level grouping before any processing: all frames of a patient, across views and augmented copies, stay in the same fold. Splits are shared across every condition and committed to the repository.
- **Augment inside the training fold only.** Standard transformations may include rotation, flipping, brightness/contrast jitter. For the optional segmentation models trained on CAMUS, the provided perturbation pipeline may additionally be used to generate additional image-quality variations.
- **Required comparison.** Report the downstream performance under the relevant processing conditions (none / classical / deep / classical + deep) using identical patient-level folds and an identical downstream prediction stage, with mean and confidence interval across folds. Report the mean and confidence interval across folds and discuss which differences exceed fold-to-fold variability.

5 Deliverables and Evaluation

5.1 Deliverables

Intermediate delivery: November 4, 2026, 23:59
This delivery must include, at a minimum, the complete noise identification and image restoration pipeline. You may also include any additional steps completed beyond this point.

Submit one `.zip` file per group on WeBeep, named with all group members' "Codice Persona" joined by underscores (e.g., `12345678_13245867.zip`). Choose one of the following formats:

- **Code-only:** the code serves as the report, so code and comments must clearly explain your implementation and reasoning.
- **Code + text report:** the report explains your reasoning, and the code needs only ordinary comments.

In both cases the code can be one or more notebooks and/or Python files, organized as you prefer. Do not include in-house images or restored frames in the archive.

Required file `answers.csv`, at the top level of the archive (not inside a subfolder).
This file is **not a challenge**: the accuracy of the tags will **not** be graded, and groups are not compared on it. It complements your text and code reports, showing what your method concludes on each image, so that we can better follow and discuss your approach. For this reason, it must be generated by your code from the released data (with the data folder given as a configurable path), containing the information listed below:

- one row for each image listed in `inhouse/labels.csv` (all splits), identified by its `image_id` (without extension);
- comma-separated, with exactly this header:
  `image_id,gaussian_blur,contrast,speckle,gaussian_noise,salt_pepper`
- every perturbation cell contains `none`, `mild`, `moderate` or `severe`, with no empty cells, so that the file can be read automatically. Uncertain or borderline cases should still be tagged with one of the listed options, and are worth discussing in the report.

An image may have **no synthetic perturbation**: this is a valid answer, reported as `none` in all five columns. "Clean" refers only to the five synthetic perturbations listed above: every ultrasound image naturally contains some speckle, and sharpness and contrast also vary from image to image. The clean CAMUS images, together with the variability across the in-house images themselves, show what this natural appearance looks like. For this reason, we expect clean images and the mildest perturbations to be the hardest to tell apart: some overlap between them is normal, and these borderline cases are good material to discuss in the report.

Example CSV rows (fictitious IDs):
`examX_2CH_ED_d0,none,moderate,mild,none,none` (contrast and speckle)
`examY_4CH_ED_d0,none,none,none,none,none` (clean)

5.2 Evaluation

The project grading is divided into two components:

- **Mandatory Classical Core (14 points):** Designed to fit the allocated project hours.
- **Optional Deep Component (4 points):** Completing this section can earn you bonus points on the final exam.

The project simulates a real-world clinical scenario to show how image preprocessing impacts downstream machine learning tasks. You will first apply classical noise-removal techniques, and then evaluate how those preprocessing choices affect a machine learning model.

Important: You will *not* be evaluated on achieving the highest possible final model metrics. Please do not spend excessive time fine-tuning the model just to bump up accuracy. Instead, your evaluation will focus entirely on:

- Your methodological approach and problem-solving process.
- The quality of your critical observations.
- Your ability to explain how and why your chosen preprocessing techniques influenced the later stages of the machine learning pipeline.

6 Requirements

- A background in biomedical engineering, computer science or a related discipline.
- Classical image-processing fundamentals: filtering, noise reduction, morphology, thresholding, region growing, active contours.
- **Python is the only supported language.** Lessons and technical support use Python (`NumPy`, `SciPy`, `scikit-image`, `OpenCV`, and `PyTorch` for the optional deep component). MATLAB is permitted but unsupported.
- Familiarity with Google Colab or Jupyter, and with handling ephemeral storage via cloud drives.
- Ability to read scientific literature, adapt code, and analyse results quantitatively.

For more information, contact us at bioimagesproject@gmail.com. The listed requirements describe the project, not strict prerequisites: if you are motivated and eager to learn, reach out regardless of prior experience.

References

[1] S. E. Petersen et al., "Reference ranges for cardiac structure and function using cardiovascular magnetic resonance (CMR) in Caucasians from the UK Biobank population cohort" *Journal of Cardiovascular Magnetic Resonance*, 19.1 (2017): 18, 2016. https://doi.org/10.1186/s12968-017-0327-9

[2] S. Leclerc et al., "Deep Learning for Segmentation Using an Open Large-Scale Dataset in 2D Echocardiography," *IEEE TMI*, 2019. https://www.creatis.insa-lyon.fr/Challenge/camus/

[3] O. Ronneberger, P. Fischer, T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI 2015. https://arxiv.org/abs/1505.04597

[4] P. Iakubovskii, *Segmentation Models PyTorch*. https://github.com/qubvel/segmentation_models.pytorch

[5] L. Chen et al., "Simple Baselines for Image Restoration," ECCV 2022. https://arxiv.org/abs/2204.04676

[6] C. B. Burckhardt, "Speckle in ultrasound B-mode scans," *IEEE Trans. Sonics Ultrason.*, SU-25(1):1–6, 1978.

[7] C. Perreault, M.-F. Auclair-Fortier, "Speckle Simulation Based on B-Mode Echographic Image Acquisition Model," CRV 2007, pp. 379–386. https://doi.org/10.1109/CRV.2007.61

[8] P. Perona, J. Malik, "Scale-space and edge detection using anisotropic diffusion," *IEEE TPAMI*, 12(7):629–639, 1990.

[9] Y. Yu, S. T. Acton, "Speckle reducing anisotropic diffusion," *IEEE TIP*, 11(11):1260–1270, 2002.

[10] A. Buades, B. Coll, J.-M. Morel, "A non-local algorithm for image denoising," CVPR 2005.

[11] R. M. Lang et al., "Recommendations for Cardiac Chamber Quantification by Echocardiography in Adults," *J Am Soc Echocardiogr*, 28(1):1–39, 2015.

[12] J. M. Bland, D. G. Altman, "Statistical methods for assessing agreement between two methods of clinical measurement," *Lancet*, 327(8476):307–310, 1986.

[13] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," ICCV 2017. https://arxiv.org/abs/1610.02391 — code: https://github.com/jacobgil/pytorch-grad-cam

[14] M. Christensen, M. Vukadinovic, N. Yuan, D. Ouyang, "Vision–language foundation model for echocardiogram interpretation," *Nature Medicine*, 30:1481–1488, 2024. https://doi.org/10.1038/s41591-024-02959-y — code: https://github.com/echonet/echo_CLIP — weights: https://huggingface.co/mkaichristensen/echo-clip

[15] Holste G, Oikonomou EK, Tokodi M, Kovács A, Wang Z, Khera R. Complete AI-Enabled Echocardiography Interpretation With Multitask Deep Learning. *JAMA*. 2025;334(4):306–318. doi:10.1001/jama.2025.8731

[16] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, July 2017. [Online]. Available: https://arxiv.org/abs/1608.03981.

[17] J. Tachella *et al.*, "DeepInverse: A Python package for solving imaging inverse problems with deep learning," *Journal of Open Source Software*, vol. 10, no. 115, p. 8923, 2025. [Online]. Available: https://doi.org/10.21105/joss.08923